Software Requirements Specification (SRS)

Hostel Permission Management System (HPMS)

Version 1.0

Document Date: 04/11/2025

Table of Contents

1. Introduction
    - 1.1 Purpose
    - 1.2 Project Scope
    - 1.3 Definitions, Acronyms, and Abbreviations
    - 1.4 Intended Audience
2. Overall Description
    - 2.1 Product Perspective
    - 2.2 Product Features
    - 2.3 User Classes and Characteristics
    - 2.4 Operating Environment
    - 2.5 Design and Implementation Constraints
    - 2.6 Assumptions and Dependencies
3. System Features (Functional Requirements)
    - 3.1 User Authentication and Profile Management
    - 3.2 Permission Request Management (Student)
    - 3.3 Permission Approval Management (Warden)
    - 3.4 Administrator Management
    - 3.5 Security and Gate Management
    - 3.6 Notification System
4. Non-Functional Requirements
    - 4.1 Performance
    - 4.2 Security
    - 4.3 Usability
    - 4.4 Reliability
    - 4.5 Responsiveness
    - 4.6 Maintainability
5. Appendices
    - 5.1 System Process Flows (Use Case Diagrams)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the requirements for the Hostel Permission Management System (HPMS). The purpose of this system is to automate and streamline the entire process of hostel students applying for, and receiving permission for outings or home visits from the college hostel authorities (wardens and chief warden). It will replace the current manual, paper-based system, providing a transparent, efficient, and auditable digital platform for students, hostel wardens, security staff, and the college administration.

1.2 Project Scope

The HPMS will be a web and mobile-based application covering the full lifecycle of a permission request:

- **Student:** Registration, profile management, permission request , and real-time tracking of request status.
- **Warden:** Secure dashboard to review, approve, or reject student requests, and manage cancellations.
- **Security:** A mobile application to verify approved permissions via a unique QR code and to record student entry and exit times.
- **Administration:** A panel to manage system users (e.g., wardens, and chief wardens) and view system-wide reports.
- **Parents:** Receive automated notifications regarding their ward's approved permissions.

1.3 Definitions, Acronyms, and Abbreviations

- **HPMS:** Hostel Permission Management System
- **SRS:** Software Requirements Specification
- **UI:** User Interface
- **USN:** University Seat Number
- **ISN:** Institution Serial Number
- **QR Code:** Quick Response Code
- **Admin:** System Administrator
- **Warden:** Hostel staff responsible for approving permissions.
- **Student:** A resident of the hostel.
- **Security:** Gate/Security staff responsible for verifying entry/exit.
- **API:** Application Programming Interface

1.4 Intended Audience

This document is intended for project stakeholders, including the development team (designers, developers, testers), project managers, and the college administration (principal, chief wardens, IT cell), to ensure a common understanding of the system being built.

2. Overall Description

2.1 Product Perspective

The HPMS is a self-contained system that will replace the existing manual logbook and paper-based (permission-slip) process. It will operate as a web-based application for Students, Wardens, and Admins, and will include a dedicated mobile application for Security Staff for QR code scanning. It will be built on a ReactJS frontend with a **custom backend application (e.g., Node.js, Python, Java). This backend will be developed to run on local servers (laptops) and subsequently deployed to a college-hosted server for production.**

2.2 Product Features

The system will provide the following key features:

- Secure user registration and login for all user roles.
- A comprehensive student profile management system.
- An intuitive form for students to submit outing/leave requests.
- A dashboard for wardens to manage and action all incoming requests.
- A QR code generation system for approved permissions.
- A mobile scanner for security to log student exit and entry.
- An automated notification system via SMS/Email/In-app.
- An admin panel for user management and system oversight.

2.3 User Classes and Characteristics

- **Student:** (Primary User)
    - Has a unique College ID (USN/ISN).
    - Can create and manage their own profile.
    - Can submit, track, and re-apply with the same permission Id for permissions
    - **Chief Warden:** Acts as the lead administrator with authority over process integrity, user assignments, and operational monitoring.
    - **Warden** (Primary User)**:** Manages a specific set of students/blocks. - Automated Mapping of Students to Wardens: When adding a new student, automatically prompt for their room number and block. Based on this information, assign a default warden and store all details in the database for future reference.
    - Has the authority to approve, reject, or cancel permissions.
    - Must be able to view detailed student information to make decisions.
- **Administrator (Admin):**
    - Has oversight of the entire system.
    - Can manage (add, update, remove) warden accounts.
    - Can access reports and archives.
- **Security Staff:**
    - Operates at the campus gate.

    - Uses a mobile device to scan QR codes.
    - Does not have access to the main web dashboard, only the scanning application.
- **Parent:**
    - A passive user who receives notifications.
    - Does not have a login to the system but can interact by contacting the Warden.

2.4 Operating Environment

- **Web Application (Student, Warden, Admin):** Will be accessible on all modern web browsers (Chrome, Firefox, Safari, Edge) on desktop, laptop, and mobile devices.
- **Backend:** A custom server-side application hosted initially on **local servers (laptops) for development and finally on a dedicated college server** for production.
- **Mobile Application (Security):** An Android application (or cross-platform) for the tablet/device used at the gate.

2.5 Design and Implementation Constraints

- **Frontend Technology:** React JS, HTML, and CSS
- **Backend Technology:** A custom server-side application (e.g., Node.js, Python/Django, Java/Spring) to be hosted on the college server.
- **Database:** A self-hosted database (e.g., MySQL, PostgreSQL, MongoDB) managed on the college server.
- **Messaging API:** Requires integration with an SMS/Email service (e.g., Twilio, Contineo).
- **External Database:** The system may need to integrate with an existing institutional database to fetch student data via USN/ISN.

2.6 Assumptions and Dependencies

- **Dependency:** An external Student Database API is available for fetching student data.
- **Dependency:** An SMS/Email gateway API will be procured for sending notifications.
- **Assumption:** All users (students, wardens) have access to a device with an internet connection and a modern web browser.
- **Assumption:** Security staff will be provided with a dedicated mobile device (tablet/phone) with a camera for scanning.
- **Assumption:** The college will provide and maintain the server infrastructure (hardware, network, OS, database) for the production environment.

3. System Features (Functional Requirements)

3.1 User Authentication and Profile Management

- **FR-3.1.1 (Multi-Factor Login):** Users shall be able to sign up and log in using a standard email/password combination.

- **FR-3.1.2 (Password Confirmation):** The signup process for both Students and Wardens must include a "Confirm Password" field, which must be validated.
- **FR-3.1.3 (Password Recovery):** A "Forgot Password" functionality shall be available, allowing users to reset their password via a secure email link or OTP.
- **FR-3.1.4 (USN Data Fetch):** On registration (or first login), the system shall attempt to fetch student data (Name, USN, Mobile, Parent's Contact, Photo) using the USN/College ID from an external institutional database.
- **FR-3.1.5 (Locked Fields):** Data fetched via USN (per FR-3.1.4) shall be non-editable by the student, with the exception of Hostel Block and Room Number.
- **FR-3.1.6 (Mandatory Profile Completion):** After the initial signup, students must be directed to a mandatory profile completion form. This form must collect:
    - Full Name
    - Phone Number
    - Year of Study
    - Academic Branch
    - Hostel Block
    - Hostel Room Number
    - User Photo Upload
- **FR-3.1.7 (Profile Settings):** Students shall have a "Profile" or "Settings" section where they can edit their profile information (fields not locked by FR-3.1.6) and change their password.
- **FR-3.1.8 (Parent Contact):** The student profile shall include a field for "Parent's Contact Number."
- **FR-3.1.9 (Dynamic Room Selection):** The profile section shall use dynamic (dependent) dropdowns for selecting Hostel Block, then Floor, then Room Number.
- **FR-3.1.10 (Room Display):** The profile shall display the complete room number in a formatted way (e.g., "M-007") after selection.
- **FR-3.1.11 (ISN/USN Logic):** The system shall store both ISN and USN. If the USN is initially empty, any update to the ISN field (or a dedicated USN field) should populate the official USN, which then becomes the primary identifier.
- **FR-3.1.12 (Username Label):** The "Username" field on the Student signup form shall be explicitly labeled "College Provided ID".

3.2 Permission Request Management (Student)

- **FR-3.2.1 (Request Form):** Students shall be able to fill out a permission request form.
- **FR-3.2.2 (Request Type):** The form must include a dropdown to select the request type:
    - Outing
    - Home Going
    - Others
- **FR-3.2.3 (Location Field):** The form must include a mandatory address/location text field, which is required only if the type "Outing" is selected.
- **FR-3.2.4 (Request Reason):** The form shall include a text area for the student to write the reason or details of the request.

- **FR-3.2.5 (Warden Selection):** The form must include a mandatory step where the student selects their assigned Warden from a predefined list. The request shall be routed to this specific warden.
- **FR-3.2.6 (Minimum Notice):** The system shall enforce a minimum notice window (e.g., 4 hours) before the requested out-time.
- **FR-3.2.7 (Weekend Timing):** For requests of type "Outing" on weekends (Sat/Sun), the default in-time shall be automatically set to 19:00 Hours.
- **FR-3.2.8 (Delay Flag):** Students shall have an option (e.g., checkbox or editable time field) to flag a delayed return and update their expected return time, which may require re-approval or notify the warden.
- **FR-3.2.9 (Reapplication Logic):**
    - If a request is **Rejected**, the student shall be allowed to reapply.
    - If a request is **Approved**, the student shall be prevented from reapplying for the exact same approved request.
- **FR-3.2.10 (Status Display):** Students shall be able to see the status of their submitted requests (Pending, Approved, Rejected) in their dashboard.

3.3 Permission Approval Management (Warden)

- **FR-3.3.1 (Warden Dashboard):** Wardens shall have a dashboard to view and manage all pending permission requests routed to them.
- **FR-3.3.2 (View Student Details):** When viewing a request, the warden must have full visibility of all the student's profile details (Name, Phone, Year, Branch, Block, Room, Photo).
- **FR-3.3.3 (Conditional Parent Contact):** The Parent's Contact Number shall *only* be visible to the warden if the student has selected the request type "Home Going."
- **FR-3.3.4 (Approve/Reject):** Wardens shall have the ability to "Approve" or "Reject" a request.
- **FR-3.3.5 (Rejection Reason):** When a Warden rejects a request, they **must** enter a reason for the rejection. This reason shall be sent to the student.
- **FR-3.3.6 (Parent Cancellation):** Wardens shall have a function to mark an approved request as "Cancelled by Parent." This is used when a parent calls the warden to cancel a trip.
- **FR-3.3.7 (Approved Archive):** Wardens and Admins shall have access to a distinct section or "Archive" that lists all historically approved permissions for record-keeping and verification.

3.4 Administrator Management

- **FR-3.4.1 (Warden CRUD):** The System Administrator shall have a management panel to Add, Update, and Remove Warden accounts.
- **FR-3.4.2 (Warden List Sync):** The list of wardens managed by the Admin (per FR-3.4.1) shall be the same list presented to students during request submission (per FR-3.2.5).

3.5 Security and Gate Management

- **FR-3.5.1 (QR Code Generation):** Upon a request being "Approved" by the warden, the system shall generate a unique QR code associated with that specific permission.

- **FR-3.5.2 (QR Code Access):** The student shall be able to access this QR code on their mobile device.
- **FR-3.5.3 (Security Scanning App):** Security staff shall have a mobile application to scan the student's QR code.
- **FR-3.5.4 (Log Out-Time):** Upon a successful scan at exit, the system shall log the student's out-time.
- **FR-3.5.5 (Log In-Time):** Upon returning, the student shall present the same QR code to be scanned, and the system shall log the in-time, completing the leave record.
- **FR-3.5.6 (Gate Verification Display):** The security/watchman staff (via the scanner or a simple view) shall be able to see the approval status (e.g., a green tick mark) and the exact Date and Time of Approval for verification.
- **FR-3.5.7 (RFID/Bus Exception):** If "Travelling via College Bus" (FR-3.2.9) was selected, the system shall bypass the need for a QR code scan and the user must obtain permission directly via a letter.

3.6 Notification System

- **FR-3.6.1 (Student Notifications):** Automated in-app, SMS, or email notifications shall be sent to the student upon:
    - Request Submission
    - Request Approved (with green tick icon and timestamp)
    - Request Rejected (must include the reason from FR-3.3.5)
    - Request Updated
- **FR-3.6.2 (Parent Notification):** An automated SMS shall be sent to the Parent's Contact Number immediately after a request is approved.
- **FR-3.6.3 (Warden Contact in SMS):** The SMS sent to the parent (FR-3.6.2) must include the Warden's contact number to facilitate the parent-cancellation flow (FR-3.3.6).
- **FR-3.6.4 (Warden Alert):** The system shall alert the Warden if a student flags a delayed return (per FR-3.2.8).

4. Non-Functional Requirements

4.1 Performance

- **NFR-4.1.1:** The web application pages shall load in under 3 seconds on the college LAN/Wi-Fi.
- **NFR-4.1.2:** The QR code scan and verification process shall take less than 2 seconds.
- **NFR-4.1.3:** The system shall support concurrent use by at least 50% of the total hostel student population (for requests) and all wardens/security staff simultaneously.

4.2 Security

- **NFR-4.2.1:** All user passwords must be hashed and salted in the database.

- **NFR-4.2.2:** The system must implement Role-Based Access Control (RBAC) to ensure students cannot access warden/admin functions, and wardens cannot access admin functions.
- **NFR-4.2.3:** All data transmission shall be over HTTPS (SSL), requiring a certificate to be installed on the college server.
- **NFR-4.2.4:** QR codes should be single-use (one for exit, one for entry) or time-bound to prevent misuse.

4.3 Usability

- **NFR-4.3.1:** The UI shall be clean, intuitive, and easy to navigate for all user roles.
- **NFR-4.3.2:** Clear visual indicators (e.g., colors, icons like a green tick) shall be used for request statuses (Pending, Approved, Rejected).
- **NFR-4.3.3:** All form fields shall have clear labels and validation messages.

4.4 Reliability

- **NFR-4.4.1:** The system shall have an uptime of 99.9% (dependent on the college server's reliability).
- **NFR-4.4.2:** All permission records shall be backed up regularly, and the system must ensure no data loss. (Database backup procedures must be established on the college server).

4.5 Responsiveness

- **NFR-4.5.1:** The web application (for Students, Wardens, Admins) must be fully responsive and functional on all common devices, including desktops, laptops, tablets, and smartphones (Android & iPhone).

4.6 Maintainability

- **NFR-4.6.1:** The code shall be well-commented and follow React and backend best practices (e.g., component-based architecture, RESTful API design) for easy maintenance and future updates.

5. Appendices

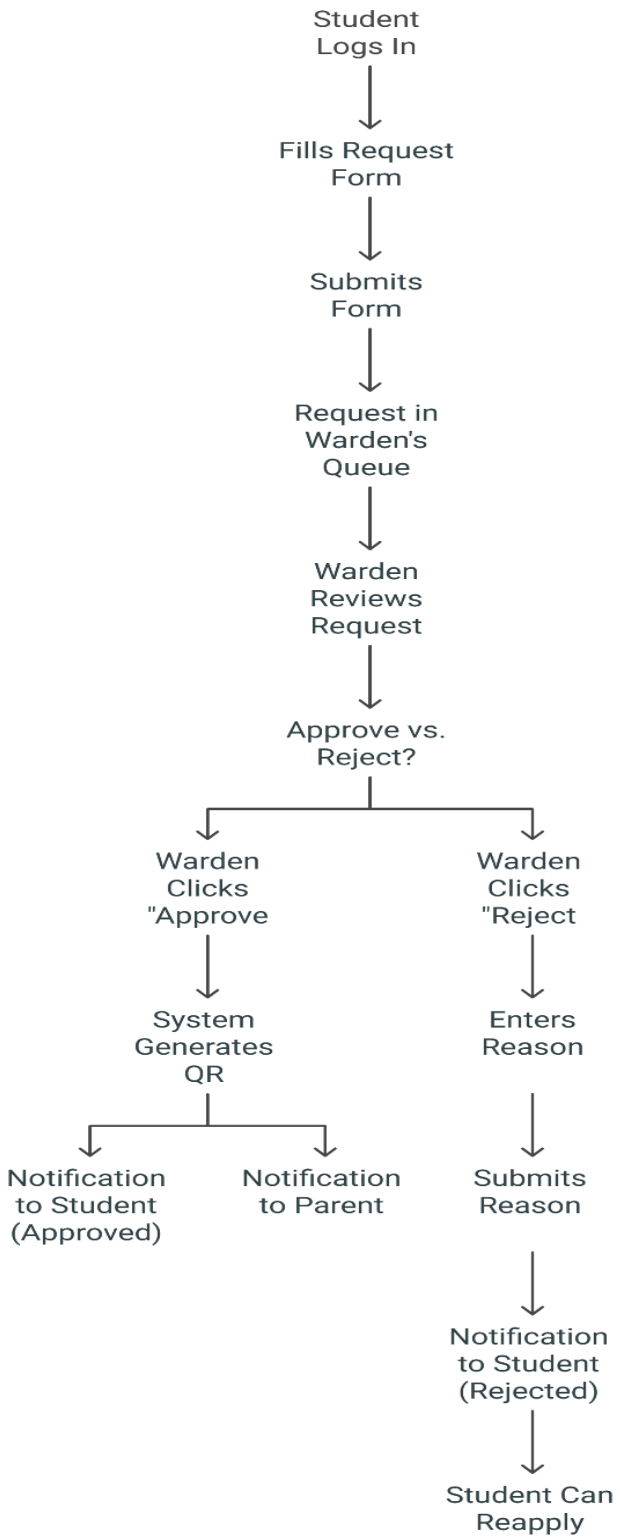
5.1 System Process Flows (Use Case Diagrams)

This is the section where you should place your flow charts and diagrams.

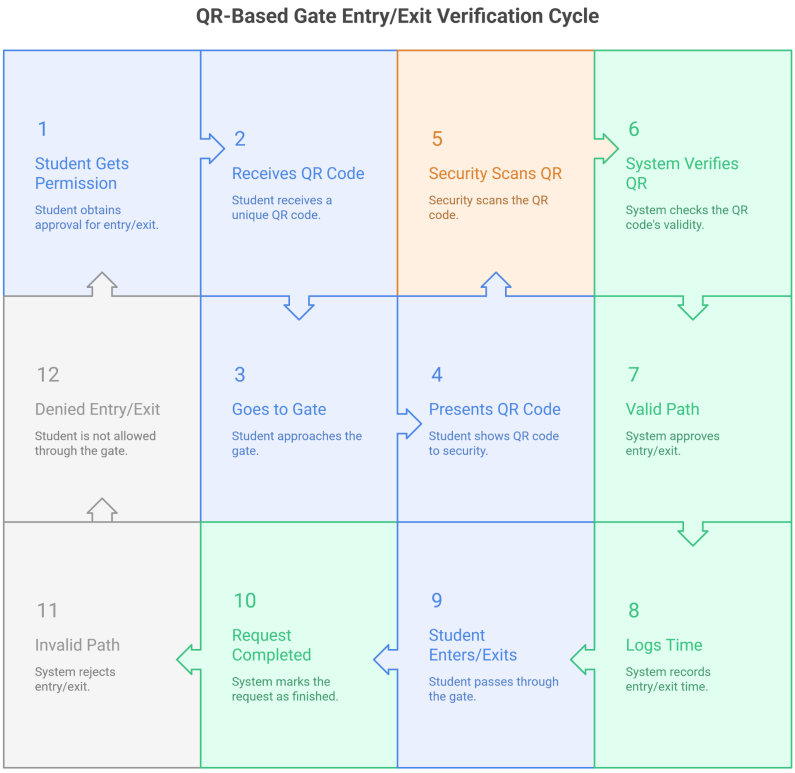
These diagrams visually represent the logic defined in the functional requirements (Section 3). You should create diagrams for at least the following processes:

- **Flow Chart 1: Student Permission Request Lifecycle**

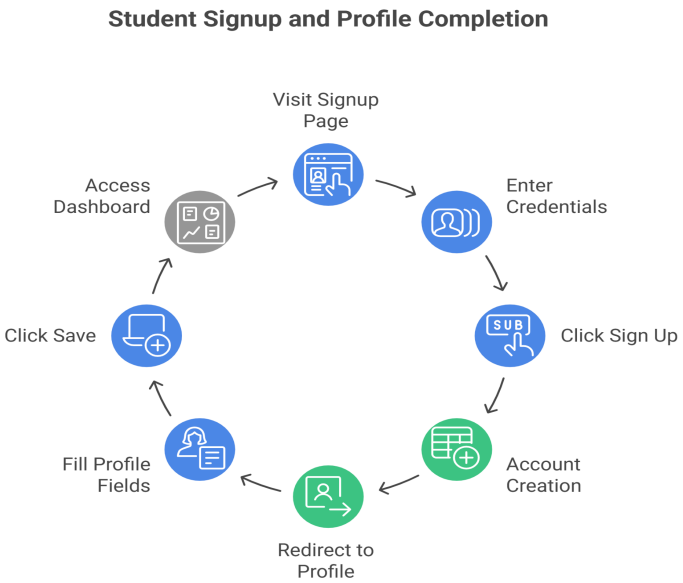
Student Permission Request Lifecycle



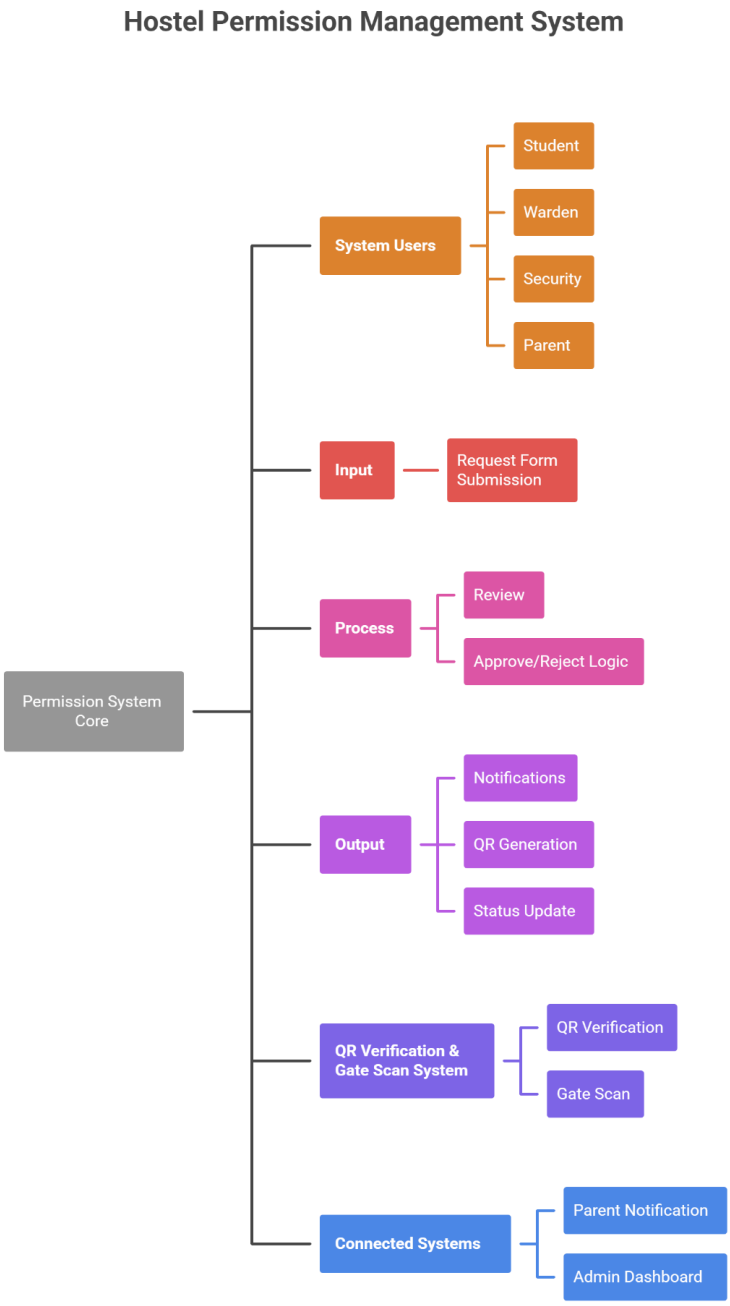
- **Flow Chart 2: Gate Verification Process**



- **Flow Chart 3: New User Registration & Profile Completion**



Flow Chart 4: Hostel Permission Management System



Hostel Permission Management System ERD

